LAPORAN PRAKTIKUM

Struktur Data

“QUEUE”

DISUSUN OLEH:

ABDUL JABBAR

2511533021

DOSEN PENGAMPU:

Dr. WAHYUDI, S.T, M.T

ASISTEN PRAKTIKUM:

SHERLY SUKMADIRA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur saya panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya saya dapat menyelesaikan makalah yang berjudul *"Queue (Antrian) dalam Struktur Data"* dengan baik dan tepat waktu.

Makalah ini disusun dengan tujuan untuk menambah wawasan dan pemahaman mengenai konsep *queue* atau antrian dalam ilmu struktur data. Queue merupakan salah satu struktur data yang penting dalam dunia informatika, yang bekerja dengan prinsip *First In First Out* (FIFO), di mana data yang masuk lebih dahulu akan diproses terlebih dahulu. Konsep ini banyak diterapkan dalam berbagai sistem, seperti sistem antrean pada pelayanan publik, pengolahan data komputer, hingga manajemen proses dalam sistem operasi.

Saya menyadari bahwa dalam penyusunan makalah ini masih terdapat kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, saya sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, saya berharap makalah ini dapat memberikan manfaat bagi pembaca dan menambah pengetahuan khususnya di bidang struktur data.

Abdul Jabbar

Padang Senin,27 April 2026

DAFTAR ISI

KATA PENGANTAR ii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 1
BAB II PEMBAHASAN 2
2.1 Pengertian 2
2.2 Praktikum pekan 4 2
2.2.1 Class QueuArray_2511533021 2
2.2.2 Class QueuArrayDriver_2511533021 4
2.2.3 Class QueuLinkedList_2511533021 5
2.2.4 Class ReverseData_2511533021 6
2.2.3 Class IterasiQueue_2511533021 8
BAB III PENUTUP 9
3.1 Kesimpulan 9
3.2 Saran 10
DAFTAR PUSTAKA 10

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam perkembangan dunia teknologi informasi yang semakin pesat, pengolahan data menjadi hal yang sangat penting. Berbagai sistem komputer membutuhkan metode yang terstruktur untuk mengatur data agar dapat diproses secara efisien dan teratur. Salah satu konsep dasar dalam struktur data yang sering digunakan adalah *queue* atau antrian.

Queue merupakan struktur data yang menggunakan prinsip *First In First Out* (FIFO), yaitu data yang pertama masuk akan menjadi data pertama yang keluar. Konsep ini sangat relevan dengan kehidupan sehari-hari, seperti antrean di bank, rumah sakit, atau layanan publik lainnya. Dalam dunia komputer, queue digunakan dalam berbagai proses seperti penjadwalan tugas (*task scheduling*), manajemen memori, hingga sistem jaringan.

Pemahaman tentang queue menjadi penting bagi mahasiswa informatika karena konsep ini merupakan dasar dalam pengembangan algoritma dan sistem yang efisien. Oleh karena itu, pembahasan mengenai queue perlu dipelajari secara mendalam agar dapat diaplikasikan dalam berbagai permasalahan nyata.

1.2 Tujuan

1. Untuk memahami pengertian dan konsep dasar *queue* dalam struktur data.
2. Untuk mengetahui prinsip kerja *queue* dengan metode FIFO.
3. Untuk mempelajari jenis-jenis *queue* dalam struktur data.
4. Untuk mengetahui penerapan *queue* dalam kehidupan sehari-hari maupun dalam sistem komputer.

1.3 Manfaat

1. Menambah wawasan dan pengetahuan mengenai struktur data, khususnya *queue*.
2. Membantu memahami cara kerja sistem antrian dalam kehidupan nyata dan dunia komputer.
3. Menjadi dasar dalam mempelajari konsep struktur data lainnya yang lebih kompleks.
4. Meningkatkan kemampuan berpikir logis dan sistematis dalam menyelesaikan masalah.

BAB II
PEMBAHASAN

2.1 Pengertian

Antrian (queue) merepresentasikan struktur data linear yang unik karena operasionalnya didasarkan pada mekanisme FIFO (First In, First Out), memastikan bahwa elemen yang ditambahkan pertama akan menjadi yang pertama kali diproses dan dihapus. Konsep ini sangat umum dan dapat ditemukan dalam kehidupan sehari-hari, seperti antrian di loket tiket, antrean kendaraan, dan sistem layanan pelanggan.

2.2 Praktikum pekan 4

Pada pekan 4 praktikum mahasiswa informatika mempelajari bagaimana queue dengan menyalin codingan dan menjelaskan nya di laporan praktikum.

2.2.1 Class QueuArray_2511533021

```java
package pekan4_2511533021;

public class QueuArray_2511533021 {
    int front, rear, size;
    int capacity;
    int array[];

    public QueuArray_2511533021(int capacity) {
        this.capacity = capacity;
        front = this.size =0;
        rear = capacity - 1;
        array = new int [this.capacity];
    }
    boolean isFull (QueuArray_2511533021 queue) {
        return (queue.size == queue.capacity);
    }
    boolean isEmpty (QueuArray_2511533021 queue) {
        return (queue.size == 0);
    }
    void enqueue(int item) {
        if (isFull (this))
            return;
        this.rear = (this.rear + 1) % this.capacity;
        this.array[this.rear] = item ;
        this.size = this.size +1;
        System.out.println(item + " enqueud to queue");
    }
    int dequeue_2511533021() {
        if (isEmpty (this))
            return Integer.MIN_VALUE;
        int item = this.array [this.front];
```


```java
        if (isEmpty (this))
            return Integer.MIN_VALUE;
        int item = this.array [this.front];
```


```java
        this.front = (this.front + 1) % this.capacity;
        this.size = this.size -1 ;
        return item;
    }
    int front_2511533021 () {
        if (isEmpty (this))
            return Integer.MIN_VALUE;

        return this.array[this.front];
    }
    int rear_2511533021 () {
        if (isEmpty (this))
            return Integer.MIN_VALUE;

        return this.array[this.rear];
    }
    void display_2511533021() {
        int i;
        if (front == rear) {
            System.out.printf("\nAntrian Kosong\n");
            return;
        }
        for (i = front; i< rear; i++) {
            System.out.printf("%d <---", array[i]);
        }
        return;
    }
}
```

1. Deklarasi Atribut
   - front : menunjukkan posisi elemen paling depan
   - rear : menunjukkan posisi elemen paling belakang
   - size : jumlah elemen dalam queue
   - capacity : kapasitas maksimum queue
   - array[] : tempat penyimpanan data queue
   Queue menggunakan konsep FIFO (First In First Out).
2. Constructor
   Menginisialisasi queue dengan kapasitas tertentu.
   front dan size diatur ke 0, rear diatur ke capacity - 1.
   Array dibuat sesuai kapasitas queue.
3. Method isFull
   Digunakan untuk mengecek apakah queue sudah penuh.
   Mengembalikan nilai true jika size sama dengan capacity.
   Method isEmpty
   Digunakan untuk mengecek apakah queue kosong.
   Mengembalikan nilai true jika size sama dengan 0.

4. Method enqueue
   Digunakan untuk menambahkan elemen ke dalam queue.
   Mengecek apakah queue penuh, jika tidak maka:
   - rear digeser dengan rumus circular (modulus)
   - data dimasukkan ke array
   - size bertambah
   Menampilkan pesan bahwa data berhasil ditambahkan.

5. Method dequeue_2511533021
   Digunakan untuk menghapus elemen dari queue (bagian depan).
   Jika queue kosong, mengembalikan nilai Integer.MIN_VALUE.
   Jika tidak:
   - mengambil data dari posisi front
   - menggeser front secara circular
   - mengurangi size
   - mengembalikan nilai yang dihapus

6.Method front_2511533021
   Digunakan untuk melihat elemen paling depan tanpa menghapusnya.
   Jika kosong, mengembalikan Integer.MIN_VALUE.

7. Method rear_2511533021
   Digunakan untuk melihat elemen paling belakang.
   Jika kosong, mengembalikan Integer.MIN_VALUE.

8. Method display_2511533021
   Digunakan untuk menampilkan isi queue.
   Jika kosong, menampilkan pesan "Antrian Kosong".

Jika tidak, menampilkan elemen dari front sampai rear.

9. Konsep Circular Queue
   Queue menggunakan perhitungan modulus (%) agar indeks kembali ke awal ketika mencapai batas array.

10. Konsep FIFO
    Data yang pertama masuk akan menjadi data pertama yang keluar.

2.2.2 Class QueuArrayDriver_2511533021

```java
package pekan4_2511533021;

public class QueueArrayDriver_2511533021 {

    public static void main(String[] args) {
        QueuArray_2511533021 queue = new QueuArray_2511533021(1000);
        queue.enqueue(10);
        queue.enqueue(20);
        queue.enqueue(30);
        queue.enqueue(40);
        System.out.println("Item di depan "+ queue.front_2511533021());
        System.out.println("item paling belakang "+ queue.rear_2511533021());
        System.out.println("tampilan queue");
        queue.display_2511533021();
        System.out.println();
        System.out.println(queue.dequeue_2511533021()+ "di hapus dari queue");
        System.out.println("item di depan: "+ queue.front_2511533021());
        System.out.println("item di belakang: "+ queue.rear_2511533021());
        System.out.println("tampilan queue setelah satu data di hapus");
        queue.display_2511533021();

    }

}
```

1. Pembuatan Objek Queue
   QueuArray_2511533021 queue = new QueuArray_2511533021(1000);
   Membuat queue dengan kapasitas 1000 elemen.

2. Proses Enqueue
   queue.enqueue(10);
   queue.enqueue(20);
   queue.enqueue(30);
   queue.enqueue(40);
   Menambahkan beberapa data ke dalam queue.

3. Menampilkan Elemen Depan
   queue.front_2511533021();
   Menampilkan data yang berada di posisi paling depan.
   Menampilkan Elemen Belakang
   queue.rear_2511533021();

Menampilkan data yang berada di posisi paling belakang.

4. Menampilkan Isi Queue
   queue.display_2511533021();
   Menampilkan seluruh isi queue sebelum ada penghapusan.

5. Proses Dequeue
   queue.dequeue_2511533021();
   Menghapus satu elemen dari bagian depan queue.

6. Menampilkan Elemen Setelah Dequeue
   Menampilkan kembali elemen depan dan belakang setelah satu data dihapus.

7. Menampilkan Queue Setelah Perubahan
   queue.display_2511533021();
   Menampilkan isi queue setelah terjadi penghapusan.

8. Konsep FIFO
   Queue bekerja dengan prinsip First In First Out, sehingga data pertama yang masuk akan menjadi yang pertama keluar.

Fungsi Driver Class:Class ini digunakan untuk menguji semua method dalam queue seperti enqueue, dequeue, front, rear, dan display.

2.2.3 Class QueuLinkedList_2511533021

```java
package pekan4_2511533021;
import java.util.*;
public class QueueLinkedList_2511533021 {

    public static void main(String[] args) {
    Queue<Integer> q = new LinkedList<>();
    for (int i = 0; i<6; i++)
        q.add(i);
    System.out.println("Elemen Antrian "+ q);
    int hapus = q.remove();
    System.out.println("hapus elemen ="+ hapus);
    System.out.println(q);
    int depan = q.peek();
    System.out.println("Kepala Antrian ="+ depan);

    int banyak = q.size();
    System.out.println("Size Antrian ="+ banyak);

    }

}
```

1. Import Library
import java.util.* digunakan untuk mengakses Queue dan LinkedList dari Java Collection Framework.

2. Method main
Merupakan titik awal eksekusi program.

3. Pembuatan Queue
Queue<Integer> q = new LinkedList<>();
Membuat queue dengan tipe data Integer menggunakan LinkedList sebagai implementasinya.

4. Pengisian Data (Enqueue)
Perulangan for digunakan untuk menambahkan data dari 0 sampai 5 ke dalam queue menggunakan method add().

5. Menampilkan Queue
System.out.println("Elemen Antrian " + q);
Menampilkan seluruh isi queue.

6. Proses Remove
q.remove();
Menghapus elemen paling depan dari queue sesuai konsep FIFO.
Nilai yang dihapus disimpan dalam variabel hapus.

7. Menampilkan Queue Setelah Remove
Menampilkan queue setelah satu elemen dihapus.

8. Method peek
q.peek();
Digunakan untuk melihat elemen paling depan tanpa menghapusnya.
Method size
q.size();
Digunakan untuk mengetahui jumlah elemen dalam queue.

2.2.4 Class ReverseData_2511533021

```java
package pekan4_2511533021;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;
public class ReverseData_2511533021 {

    public static void main(String[] args) {
        Queue<Integer> q = new LinkedList <Integer>();
        q.add(1);
        q.add(2);
        q.add(3);
        System.out.println("sebelum reverse"+ q);
        Stack<Integer> s= new Stack<Integer>();
        while(!q.isEmpty()) {
            s.push(q.remove());
        }
        while (!s.isEmpty()){
            q.add(s.pop());
        }
        System.out.println ("sesudah reverse "+ q);

    }

}
```

1. Import Library
   - LinkedList digunakan sebagai implementasi Queue.
   - Queue digunakan untuk struktur data antrian.
   - Stack digunakan untuk struktur data tumpukan.

2. Method main
   Merupakan titik awal eksekusi program.
   Pembuatan Queue
   Queue<Integer> q = new LinkedList<Integer>();
   Membuat queue dengan tipe data Integer.
3. Pengisian Data Queue
   q.add(1);
   q.add(2);
   q.add(3);
   Menambahkan elemen ke dalam queue.

4. Menampilkan Data Awal
   Menampilkan isi queue sebelum dilakukan reverse.
   Pembuatan Stack
   Stack<Integer> s = new Stack<Integer>();
   Stack digunakan sebagai bantuan untuk membalik urutan data.

5. Memindahkan Data dari Queue ke Stack
while(!q.isEmpty())
Mengambil semua elemen dari queue menggunakan remove() lalu memasukkannya ke stack menggunakan push().
Proses ini membalik urutan karena stack bersifat LIFO.

6. Memindahkan Data dari Stack ke Queue
while(!s.isEmpty())
Mengambil elemen dari stack menggunakan pop() lalu memasukkannya kembali ke queue menggunakan add().
Hasilnya urutan data menjadi terbalik.

kode ini menunjukkan cara membalik isi queue dengan memanfaatkan stack sebagai struktur bantu

2.2.3 Class IterasiQueue_2511533021

```java
package pekan4_2511533021;
import java.util.*;

public class IterasiQueue_2511533021 {

    public static void main(String[] args) {
        Queue<String>q = new LinkedList<>();

        q.add("Praktikum");
        q.add("Struktur");
        q.add("Data");
        q.add("Dan");
        q.add("Algoritma");
        Iterator<String> iterator = q.iterator();
        while (iterator.hasNext()) {
            System.out.print(iterator.next()+"");
        }

    }

}

```

1. Import Library

   import java.util.* digunakan untuk mengakses Queue, LinkedList, dan Iterator

2. Pembuatan Queue

   Queue<String> q = new LinkedList<>();

   Membuat queue dengan tipe data String.

3. Pengisian Data Queue

   q.add("Praktikum");

   q.add("Struktur");

   q.add("Data");

   q.add("Dan");

   q.add("Algoritma");

Menambahkan beberapa elemen ke dalam queue.

4. Pembuatan Iterator

Iterator<String> iterator = q.iterator();

Iterator digunakan untuk menelusuri setiap elemen dalam queue.

5. Proses Iterasi

while (iterator.hasNext())

Perulangan dilakukan selama masih ada elemen dalam queue.

6. Menampilkan Data

iterator.next() digunakan untuk mengambil elemen berikutnya.

Data ditampilkan satu per satu menggunakan System.out.print().

7. Konsep Iterasi

Iterator memungkinkan pengaksesan elemen satu per satu tanpa mengubah isi queue,Kode ini menunjukkan cara menelusuri seluruh elemen dalam queue menggunakan iterator.

BAB III
PENUTUP

3.1 Kesimpulan

Pada praktikum ini telah dipelajari konsep struktur data Queue yang bekerja dengan prinsip FIFO (First In First Out), yaitu data yang pertama masuk akan menjadi data pertama yang keluar. Implementasi queue dapat dilakukan menggunakan array maupun LinkedList, serta didukung oleh Java Collection Framework melalui interface Queue.

Melalui praktikum ini, dipahami berbagai operasi dasar pada queue seperti enqueue (menambah data), dequeue (menghapus data), peek (melihat elemen depan), dan pengecekan kondisi queue seperti isEmpty dan size. Selain itu, juga dipelajari bagaimana melakukan iterasi pada queue menggunakan Iterator serta memanfaatkan Stack untuk membalik urutan data dalam queue.

Dengan adanya praktikum ini, mahasiswa dapat memahami cara kerja queue secara lebih mendalam dan mampu mengimplementasikannya dalam berbagai kasus,

seperti antrian layanan, pengolahan data, dan sistem lainnya yang membutuhkan pengaturan data secara berurutan.

3.2 Saran

Dalam praktikum selanjutnya, diharapkan mahasiswa dapat lebih memahami konsep dasar struktur data sebelum melakukan implementasi program agar tidak terjadi kesalahan dalam logika. Selain itu, mahasiswa disarankan untuk lebih sering berlatih membuat program dengan berbagai variasi kasus agar pemahaman terhadap Queue semakin baik.

Mahasiswa juga diharapkan dapat memperhatikan penulisan kode yang rapi dan terstruktur serta menambahkan komentar pada bagian penting agar program mudah dipahami. Selain itu, penggunaan error handling sebaiknya ditingkatkan agar program dapat berjalan dengan lebih aman dan tidak mengalami kesalahan saat kondisi tertentu, seperti antrian kosong atau penuh.

Terakhir, disarankan untuk mencoba mengembangkan program Queue ke dalam kasus nyata, seperti sistem antrian rumah sakit, bank, atau layanan online, sehingga pemahaman menjadi lebih aplikatif.

DAFTAR PUSTAKA

Telkom University. (2025). *Queue: Pengertian, fungsi, jenis, implementasi, dan contoh penggunaannya*. Diakses pada 27 April 2026, dari Queue: Pengertian, Fungsi, dan Jenisnya